PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

CONSTRUCCIÓN Y DOCUMENTACIÓN DE UNA API CON EXPRESS

Presentado a: Yoly Alexandra Guevara

Por Aprendiz: Luisfernando conejo quiñones

Ficha: 3312932

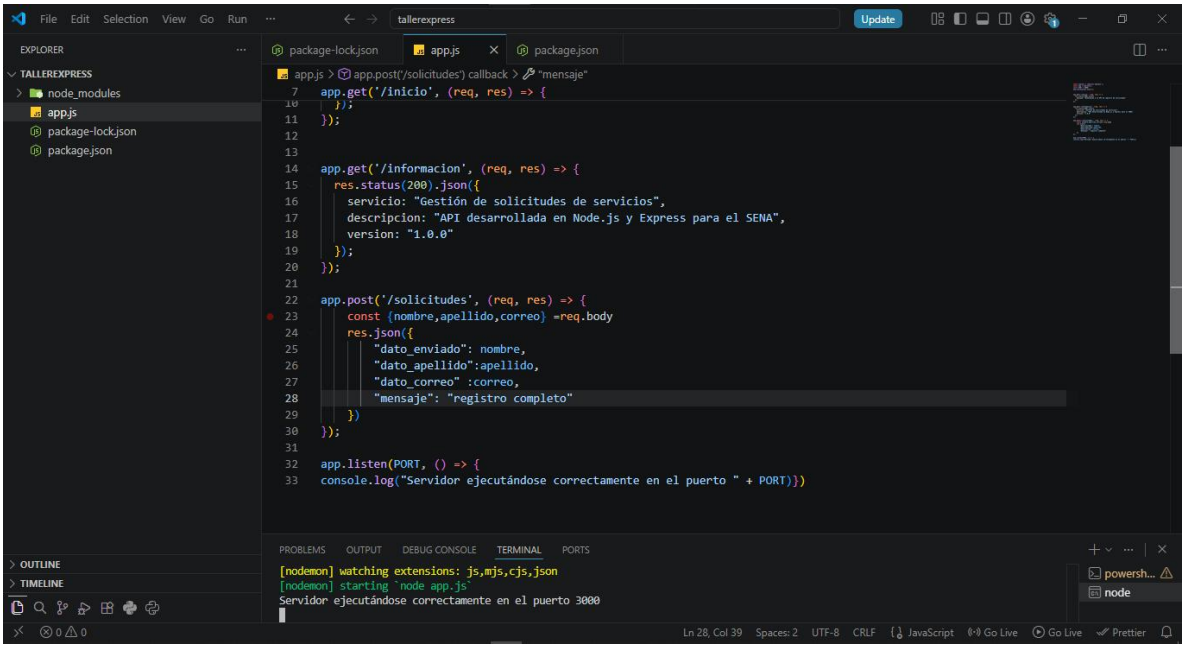
Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 20 del 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 1:

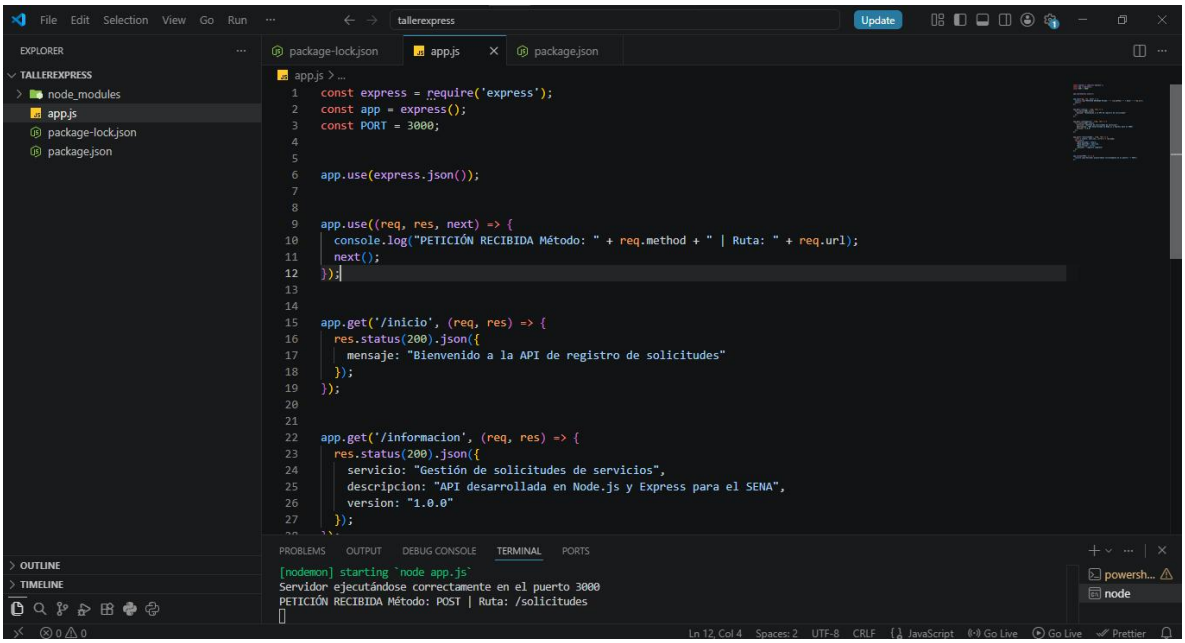
En este punto se estructuró la base de la API REST con **Node.js** y **Express**. Se habilitó el middleware express.json() para procesar datos en dicho formato, se crearon las rutas GET /inicio e /informacion para responder con la información del servicio y un código HTTP 200, y la ruta POST /solicitudes para capturar los datos enviadas desde el cuerpo de la petición (req.body) y confirmar el registro. Finalmente, el servidor queda escuchando activamente en el puerto 3000.

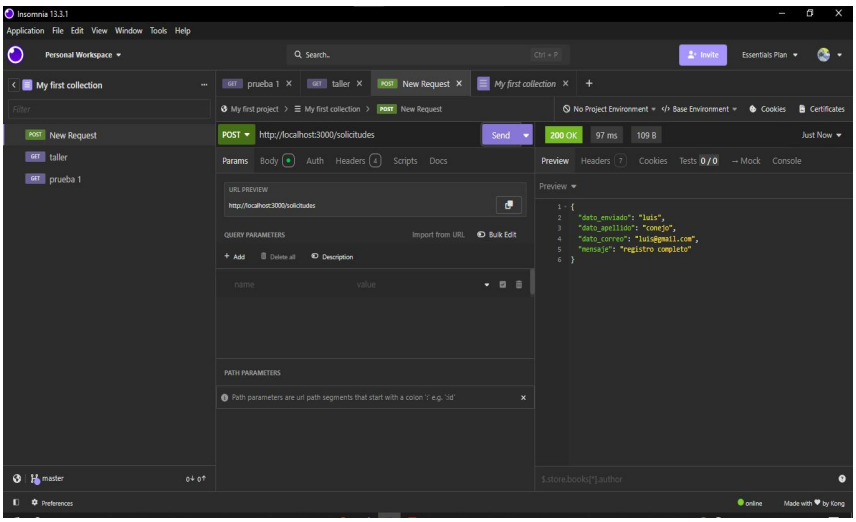


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 2: Construcción del middleware propio

Para este punto creamose e implementamos un **middleware de registro** usando app.use(), colocándolo antes de las rutas. Su trabajo es interceptar cada petición que entra, extraer el método HTTP (req.method) y la ruta (req.url), e imprimirlos en la consola. Una vez registrado el mensaje, usamos next() para darle paso libre a la petición hacia la ruta correspondiente sin bloquear el servidor



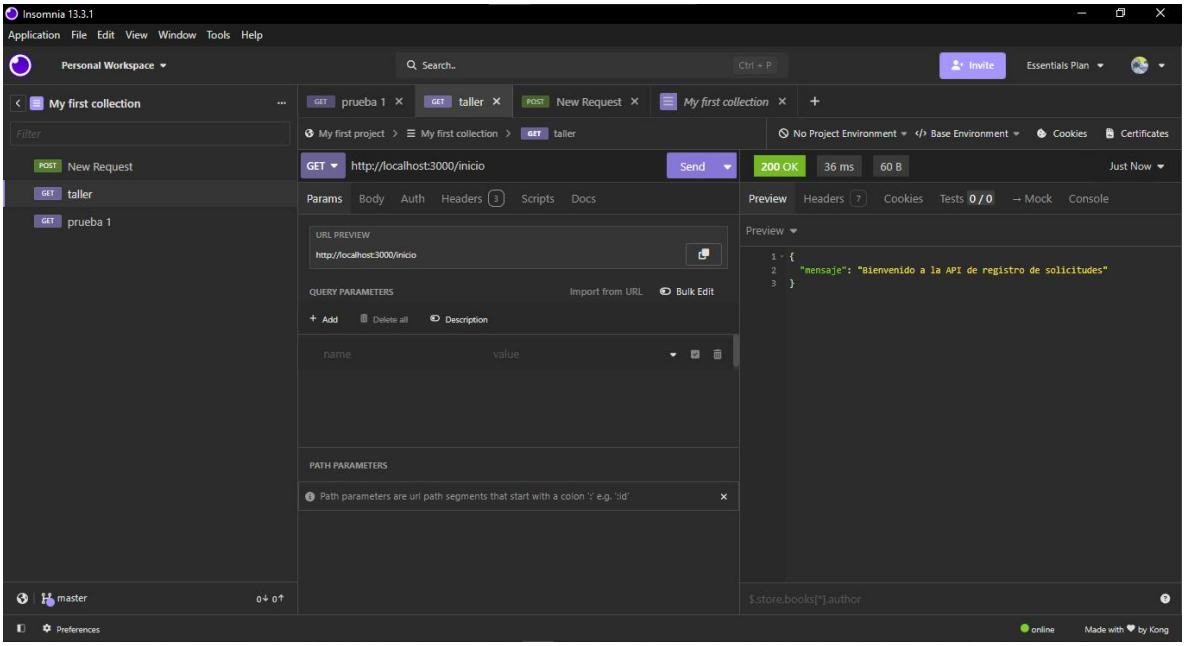


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

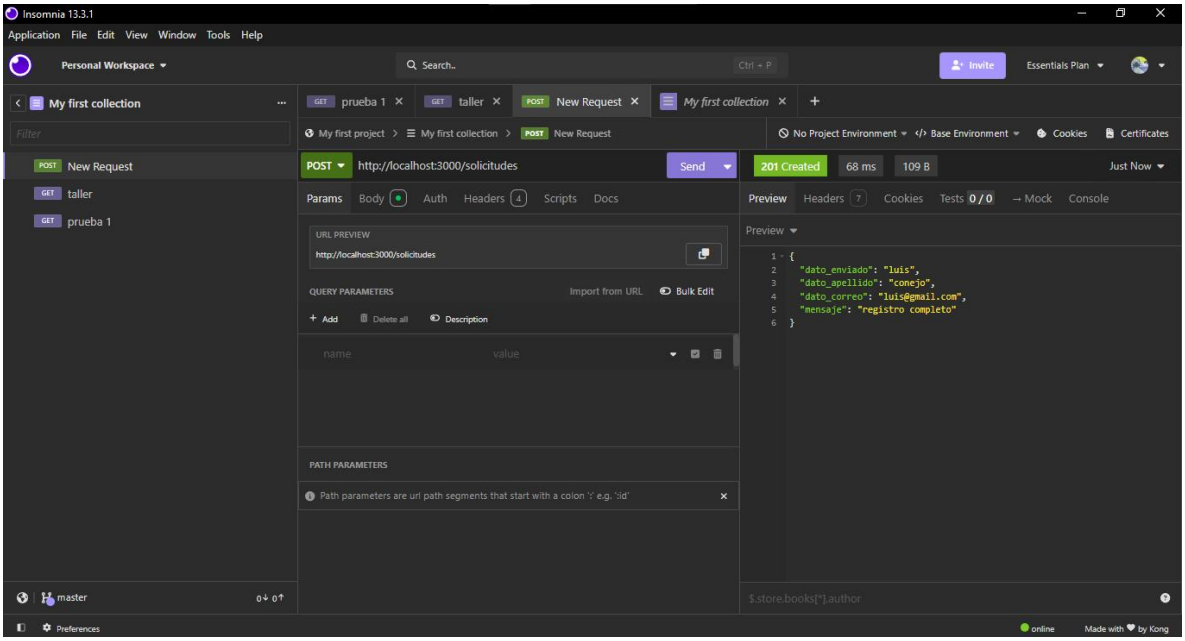
Punto 3:Códigos de estado HTTP y manejo de errores

Codigos:

200 OK: Indica éxito general en peticiones GET. Lo usamos para devolver la información de bienvenida e
institucional.

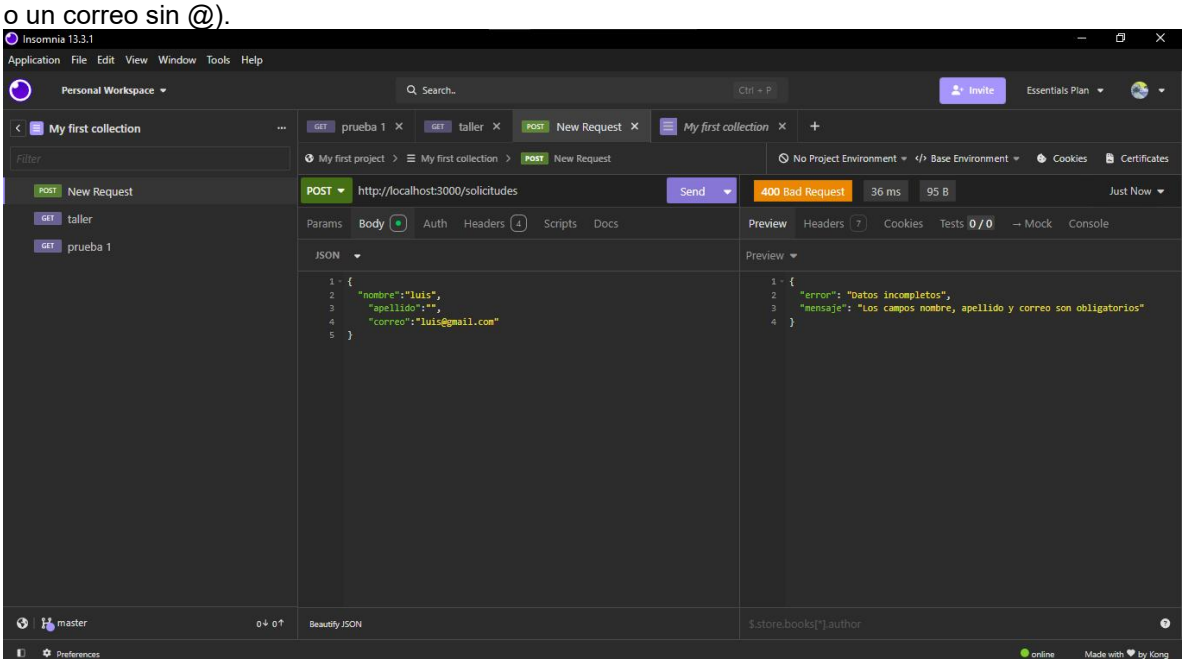


201 Created: Confirma que un recurso fue creado o procesado con éxito en peticiones POST.

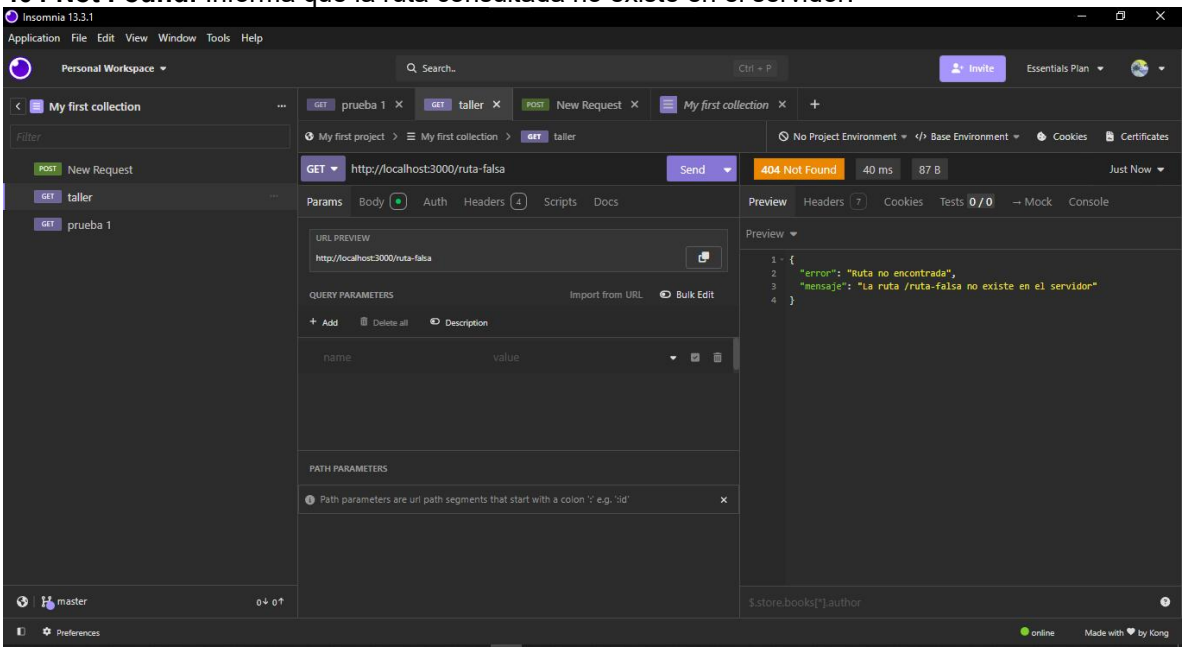


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

400 Bad Request: Avisa que el cliente envió datos incompletos o inválidos (como faltar un campo o un correo sin @).

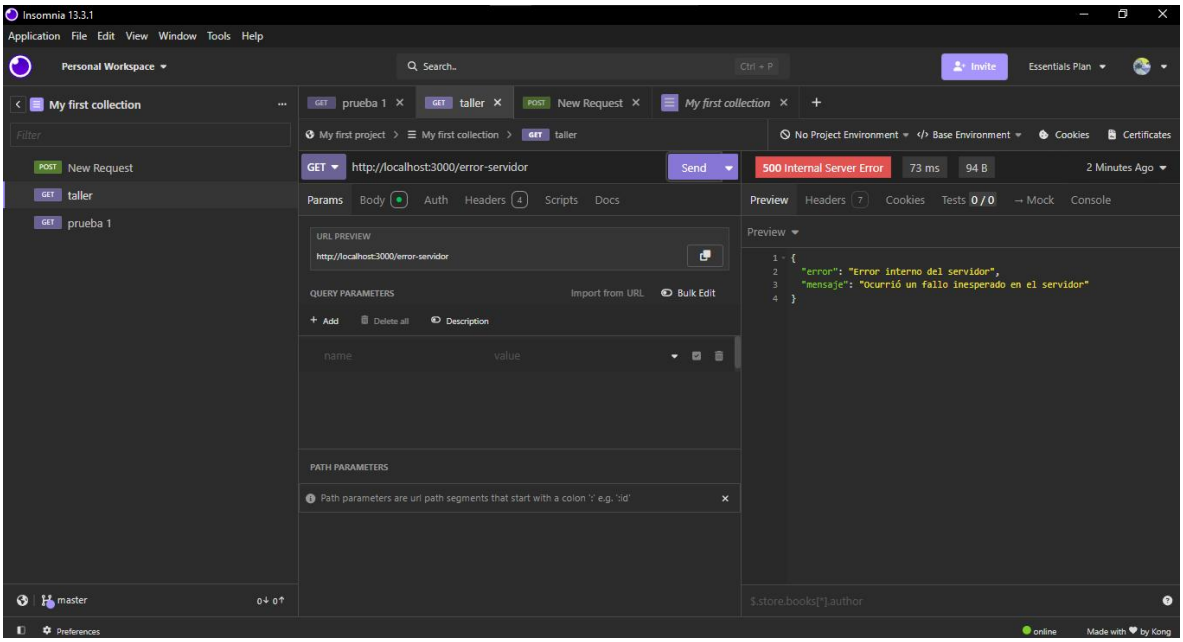


404 Not Found: Informa que la ruta consultada no existe en el servidor.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

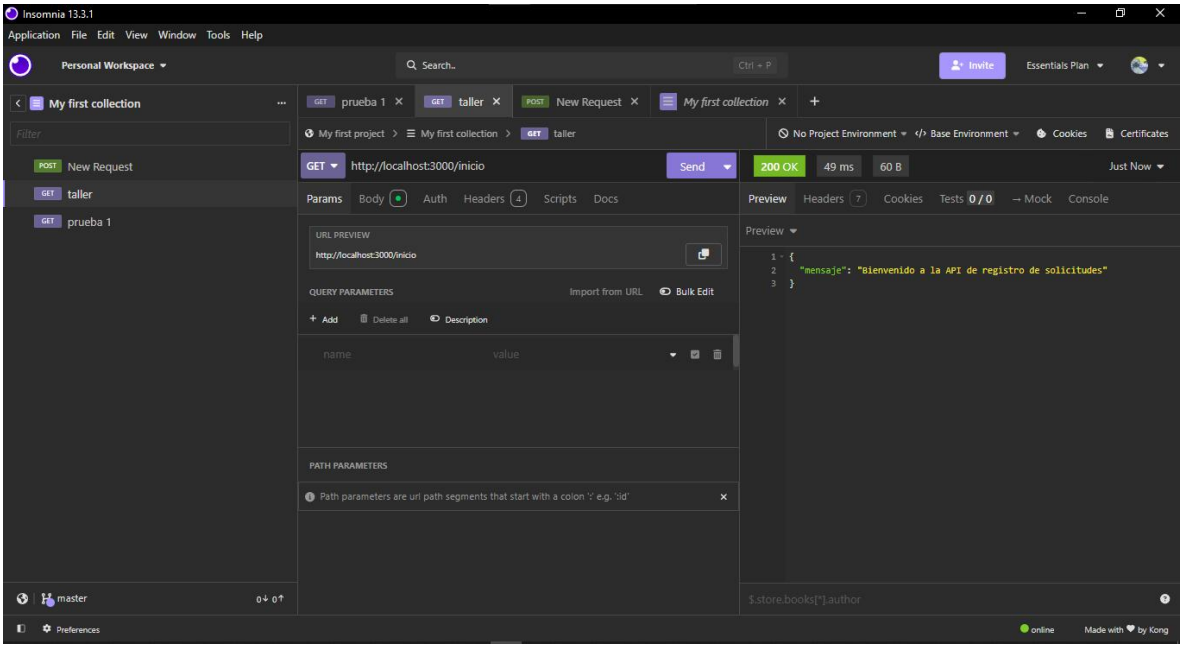
500 Internal Server Error: Indica que ocurrió un fallo inesperado del lado del servidor debido a un error interno no controlado.



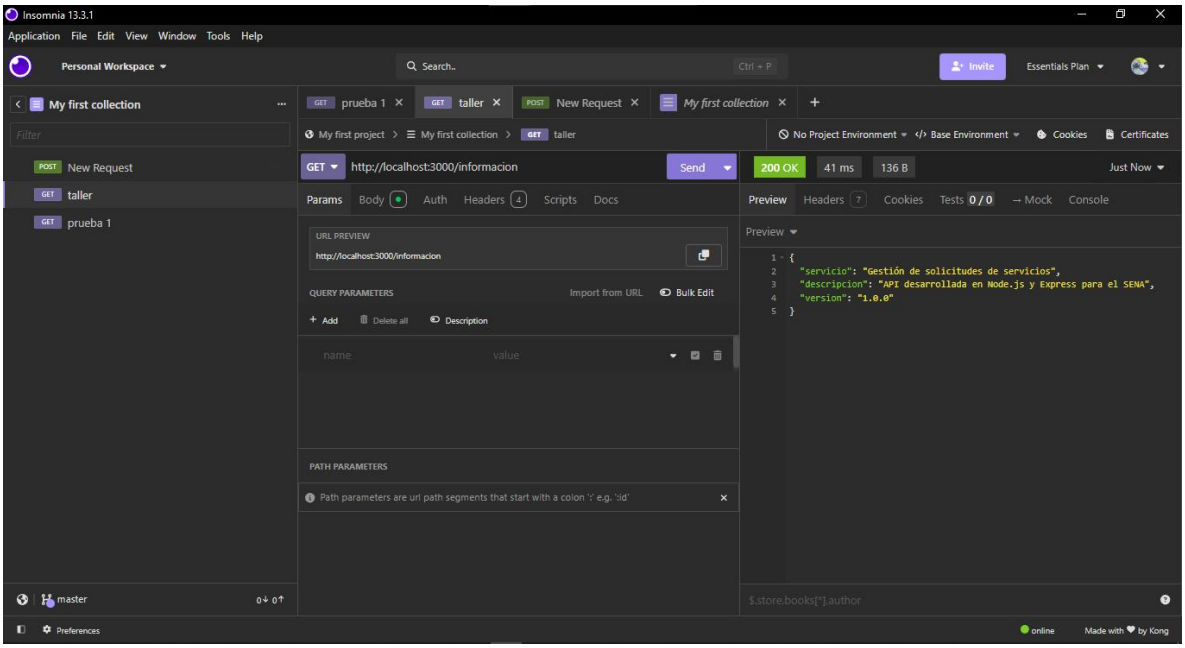
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Punto 4:Pruebas de la API

- GET /inicio: verificar que devuelve una respuesta JSON.

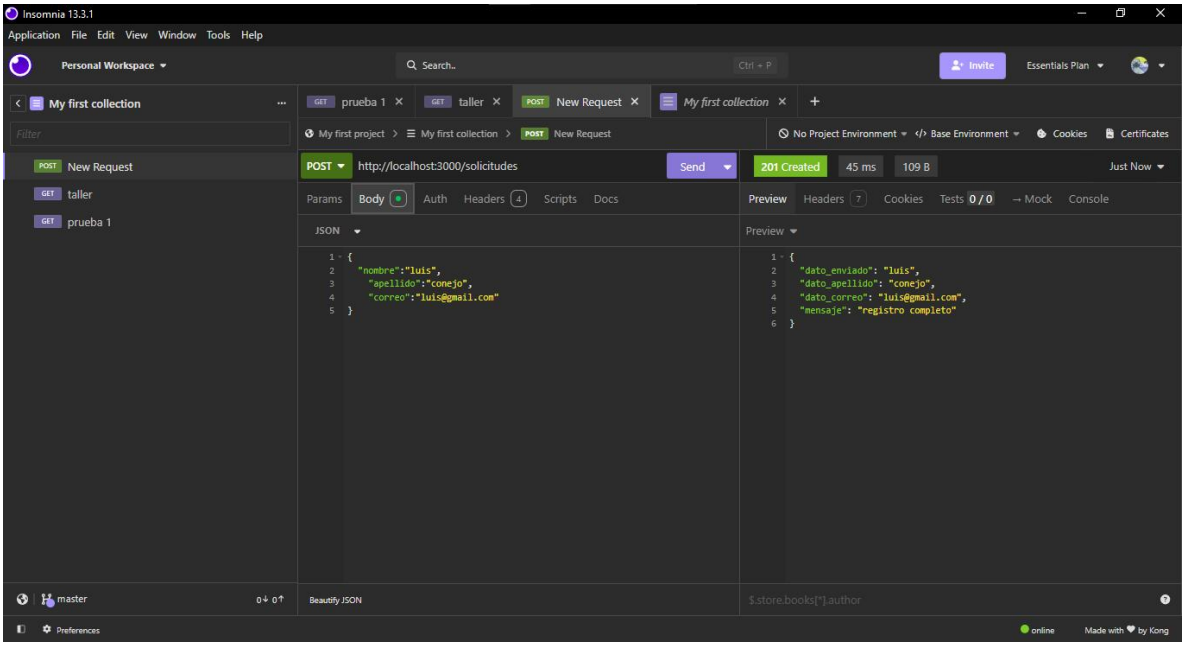


- GET /informacion: verificar que devuelve la información general.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

- POST /solicitudes con nombre, apellido y correo válidos.



- Probar qué ocurre si se envía un cuerpo JSON y explicar el papel de express.json().

Prueba y papel de express.json(): Al mandar el JSON desde Insomnia en la petición POST, la API procesa los datos sin lío gracias a que express.json() hace de traductor, convirtiendo ese texto en un objeto de JavaScript fácil de usar (req.body). Si no pusieras esa línea, el req.body llegaría vacío (undefined) y la API ni se enteraría de lo que le enviaste.

Punto 5:. Investigación obligatoria: middleware en Express

1. ¿Qué es un middleware en Express y en qué momento del ciclo de una petición se ejecuta?

Un middleware es una función intermedia que actúa como un "filtro" o "peaje" entre la petición que envía el cliente y la respuesta final que da el servidor. Se ejecuta **durante el ciclo de la petición**, justo después de que el servidor recibe la solicitud pero antes de que se envíe una respuesta o se termine de procesar la ruta.

2. ¿Qué significan los objetos req y res?

req (Request / Petición): Es un objeto que representa la solicitud realizada por el cliente. Contiene toda la información enviada, como los datos del cuerpo (req.body), los parámetros de la URL, los encabezados y el método HTTP.

res (Response / Respuesta): Es el objeto que utiliza el servidor para responderle al cliente. Permite definir el estado HTTP (como 200, 400 o 404) y enviar datos en formato JSON, texto o HTML.

3. ¿Cuál es la diferencia entre un middleware global, uno de ruta específica y uno de manejo de errores?

Middleware global: Se aplica a **todas** las peticiones que llegan al servidor (se define con app.use()). Sirve para tareas generales como registrar peticiones o parsear JSON.

Middleware de ruta específica: Se ejecuta **únicamente** cuando se consulta una ruta concreta. Se usa para tareas puntuales, como verificar si un usuario tiene acceso antes de mostrarle cierta información.

Middleware de manejo de errores: Se encarga de capturar y procesar fallos inesperados en la aplicación. A diferencia de los demás, recibe 4 parámetros: (err, req, res, next).

4. ¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía JSON?

¿Qué hace?: Lee el cuerpo de las peticiones entrantes en formato JSON y lo transforma en un objeto de JavaScript accesible mediante req.body.

¿Qué problema ocurre si no está?: El servidor recibe los datos como texto plano no procesado, provocando que req.body llegue como undefined. Esto impide leer o validar la información enviada por el cliente.

5. ¿Por qué importa el orden en que se registran los middleware y las rutas?

Porque Express ejecuta el código **de arriba a abajo**, de forma secuencial. Si colocas un middleware de validación o express.json() *después* de definir las rutas, estas no recibirán los datos procesados. Del mismo modo, si pones un middleware como el del error 404 al principio del archivo, atrapará todas las peticiones antes de que lleguen a sus rutas correspondientes.

6. ¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo?

La petición se queda **"colgada" o congelada indefinidamente** esperando una respuesta. El navegador o cliente (como Insomnia) esperará hasta que se agote el tiempo de espera (*timeout*) y lanzará un error de conexión, afectando la disponibilidad del servidor.

7. ¿Cuál es la diferencia entre un middleware y la función que atiende una ruta?

Middleware: Su objetivo principal es procesar, modificar, validar o registrar la petición y pasar el control a la siguiente función mediante next().

Función de ruta (Controlador): Es el destino final de la petición. Su función es realizar la lógica de negocio requerida y **cerrar el ciclo** enviando la respuesta definitiva al cliente con res.json() o res.send().

8. ¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite?

Ingreso de datos corruptos o inválidos: La base de datos podría guardarse con campos vacíos o formatos incorrectos.

Fallas internas en el servidor: El código intentará procesar variables inexistentes provocando errores inesperados (código 500).

Vulnerabilidades de seguridad: Usuarios no autorizados podrían acceder a rutas protegidas o modificar información de otros sin permiso.

9. Ejemplo cotidiano (sin código)

Imagina el acceso a un **discoteca o concierto**:

Punto de control 1 (Middleware Global): El personal de seguridad revisa que todos lleven su entrada válida antes de entrar al recinto. Si no la llevan, no pasan.

Punto de control 2 (Middleware de Ruta Específica): Para ingresar a la **zona VIP**, hay un segundo filtro que verifica si la entrada es de categoría VIP.

Punto de destino (Función de Ruta): Una vez superados los controles, ingresas al lugar y el barman te entrega la bebida (la respuesta final).

Punto 8:Documento técnico obligatorio: paso a paso

Paso o decisión	Por qué se realiza (Justificación)	Qué ocurriría si se omite
Instalación de Node.js y npm	Node.js nos da el entorno para ejecutar JavaScript en el servidor, y npm es el gestor que nos permite descargar e instalar librerías. **Comando:** Instalar el ejecutable oficial de Node.js.	Sin Node.js no habría cómo interpretar ni correr el código backend en la máquina. Sin npm no podríamos instalar herramientas clave como Express.
Creación de la carpeta y apertura del proyecto	**Organización:** Permite guardar y aislar todos los archivos y dependencias del proyecto en un único espacio de trabajo dentro de VS Code.	Los archivos quedarían regados por todo el disco duro, dificultando las rutas de acceso, la instalación de paquetes y el control de versiones con Git.

Inicialización del proyecto (package.json)	**Administración:** Crea el archivo package.json que registra la información del proyecto, las librerías instaladas y los scripts de arranque. **Comando:** npm init -y	Sin este archivo, npm no sabría qué dependencias usa la app. Se perderían los scripts de inicio y sería difícil replicar el proyecto en otro computador.
Instalación de Express	**Facilidad de servidor:** Es el framework que simplifica la creación del servidor, el manejo de rutas y la gestión de peticiones HTTP. **Comando:** npm install express	Habría que crear el servidor desde cero usando el módulo nativo http de Node.js, requiriendo mucho más código y siendo más difícil de mantener.
Instalación y configuración de Nodemon	**Soporte al desarrollo:** Reinicia automáticamente el servidor en la consola cada vez que guardamos un cambio en el código. **Comando:** npm install --save-dev nodemon	Tendrías que apagar y encender manualmente el servidor en la terminal (Ctrl + C y luego node index.js) tras cada cambio que hagas en el código.
Archivo principal y puerto	**Punto de entrada:** El archivo principal (index.js o app.js) arranca Express, y el puerto 3000 define la puerta de red por donde el servidor escucha. **Configuración:** app.listen(3000)	Si el servidor no escucha en un puerto activo, cualquier cliente como Insomnia o el navegador recibirá un error de conexión rechazada (ECONNREFUSED).
Middleware para analizar JSON (express.json())	**Preparación de datos:** Convierte los datos JSON enviados por el cliente en un objeto de JavaScript manipulable mediante req.body. **Configuración:** app.use(express.json())	Al intentar leer los datos en peticiones POST, req.body llegará como undefined y la API no podrá leer la información enviada por el cliente.
Middleware propio (logger)	**Monitoreo:** Muestra en la consola cada petición que llega, indicando el método HTTP y la ruta, para facilitar la depuración. **Configuración:** app.use((req, res, next) => ...)	No tendrías visibilidad en la terminal sobre qué peticiones están entrando. Si se coloca en el orden incorrecto o falta next(), se podría bloquear el flujo.
Rutas GET y POST	**Propósito:** GET se usa para solicitar o consultar información del servidor, y POST para enviar y registrar datos nuevos desde el cliente. **Configuración:** app.get(), app.post()	Si falta una ruta o su verbo HTTP, el servidor responderá con un error 404 Not Found, dejando esa función inhabilitada para el usuario.

Validación y códigos HTTP	**comunicación estandarizada:** Informan con claridad al cliente sobre el resultado usando códigos estándar como 200, 201, 400, 404 y 500. **Configuración:** res.status(400).json(...)	Responder siempre con éxito (200 OK) ocultaría errores reales, confundiendo al cliente frontend y permitiendo el ingreso de datos incorrectos a la aplicación.
Pruebas de la API	**Demostración:** Permite comprobar en Insomnia que la API responde bien tanto con datos correctos como con errores o campos faltantes. **Ejecución:** Pruebas desde Insomnia.	Pasarían inadvertidos fallos de validación, rutas mal escritas o errores del servidor que podrían hacer fallar la aplicación en producción.
Git y repositorio remoto	**Seguimiento y respaldo:** Permite guardar el historial de cambios, trabajar organizadamente y respaldar el código de forma segura en la nube mediante GitHub. **Comandos:** git init, git add ., git commit -m "...", git push	Si le pasa algo al computador, podrías perder todo el trabajo. Además, no habría forma de revisar versiones anteriores o revertir errores en el código.